

EtchGuard

이해 가이드

식각 장비 이상 징후 조기예측 시스템이 데이터를 받아 모델을 서빙하고, 성능이 떨어지면 스스로 재학습해 교체하기까지의 흐름을 그림으로 설명합니다.

판교 9반 2조 · P282 김민정 · P298 이재겸 · P287 김태동 · P289 박규리 · P280 김동욱 · P301 임동건
2026-10-01 · v3.0

차례

- | | |
|-----------------|--------------------|
| 1. 이 시스템이 푸는 문제 | 7. 드리프트 판정 |
| 2. 전체 지도 | 8. 재학습 시퀀스 |
| 3. 데이터와 시간 분할 | 9. 게이트 재검증 |
| 4. 요청 하나의 여정 | 10. 운영 지표와 로그 |
| 5. Lazy와 Eager | 11. Docker |
| 6. 버전과 배포 게이트 | 12. 코드 지도와 직접 실행하기 |
| | 13. 한계 |

읽는 법 본문에서 **※번호**가 붙은 용어는 그 절 끝의 **개념 짚기**에 뜻과 이 프로젝트에서의 쓰임을 정리했습니다. 모든 수치는 2026-10-01 최종 실행 결과(evidence/, logs/)입니다.

1 이 시스템이 푸는 문제

식각 장비는 완전히 멈추기 전에 센서가 먼저 조금씩 흔들립니다. 그런데 센서별 고정 경보는 자주 틀리고, 장비 5대 × 센서 12종 × 하루 48사이클이면 하루 측정값이 2,880개라 사람이 다 볼 수도 없습니다. 그 사이 이상 상태로 진행된 웨이퍼는 사후 계측에서야 불량으로 드러납니다.

문제	무엇이 일어나는가	EtchGuard의 답
모델을 현장에서 못 쓴다	모델이 파일로만 있으면 점검 판단에 쓰이지 않습니다.	서빙 ^{*1} : <code>/predict</code> 로 30분 뒤 이상 점수와 등급을 제공
더 나쁜 모델이 배포된다	검증 없이 바뀐 모델이 위험을 놓치거나 오경보를 늘립니다.	MLOps ^{*2} : 기록, 버전 관리, 배포 게이트
모델이 조용히 나빠진다	공정이 바뀌어도 API는 200을 돌려주고, 예측만 계속 틀립니다.	AIOps ^{*3} : 예측 오차 ^{*4} 감시, 재학습, 재검증, 교체
경보를 아무도 믿지 않는다	계절 변화나 일시적 팀까지 경보로 올리면 진짜 경보도 무시됩니다.	3회 연속 판정, 같은 달 기준 비교

* 개념 짚기

*1 모델 서빙

학습된 모델을 API로 감싸 다른 시스템이 호출할 수 있게 하는 일입니다. 노트북 안에만 있는 모델은 현장에서 쓸 수 없습니다.
EtchGuard → FastAPI의 `POST /predict` 가 모델의 입구입니다.

*3 AIOps

배포된 모델을 계속 지켜보다가 성능이 떨어지면 알리고, 스스로 재학습하고 다시 배포하는 운영 자동화입니다.
EtchGuard → 드리프트 판정 → [WARN] 알림 → 재학습 → 재검증 → 교체로 이어집니다.

*2 MLOps

모델을 기록하고 버전으로 관리하며, 검증을 통과한 것만 배포하는 체계입니다. 모델을 믿고 바꿀 수 있게 해 줍니다.
EtchGuard → MLflow 기록, Registry, 배포 게이트, Docker가 여기에 속합니다.

*4 예측 오차와 확정 점수

확정 점수는 공정이 끝난 뒤 계측으로 정해지는 실제 이상 점수(라벨)입니다. 예측과 이 값의 차이는 코드 에러 없이 모델이 틀리고 있는지 알려 주는 유일한 신호입니다.
EtchGuard → `/predict/batch-test` 로 들어와 (예측, 실제) 쌍이 됩니다.

2 전체 지도

시스템은 세 띠로 나뉩니다. 위는 오프라인에서 모델을 만들고 배포 여부를 정하는 곳, 가운데는 컨테이너 안에서 요청을 받는 서버, 아래는 예측 오차를 지켜보다가 스스로 재학습하는 AIOps입니다. 아래 띠의 결과가 다시 위의 Registry와 가운데의 ModelManager로 돌아가면서 루프가 닫힙니다.



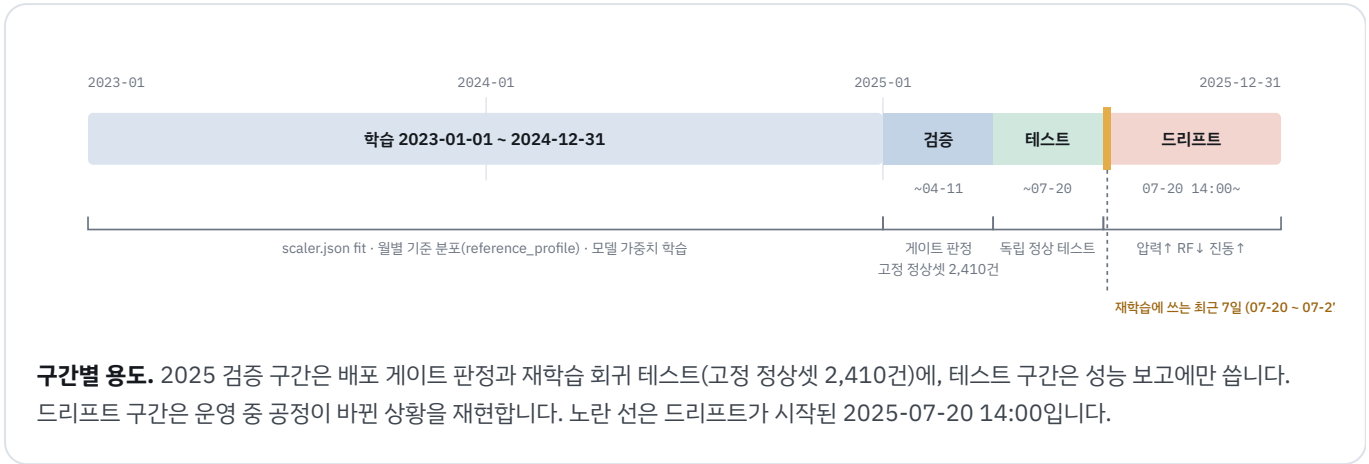
흐름별 경로

흐름	지나가는 단계	담당 코드
① 학습·배포	CSV 검증 → 시간 순 분할 → 학습 구간으로 스케일러 1회 fit → LSTM 학습(검증 RMSE 3.164) → 배포 게이트 → Production v1	<code>data.read_data</code> · <code>fit_scaler</code> · <code>train_base</code> · <code>deployment_gate</code> · <code>register_bundle</code>
② 예측 요청	콘솔이 최근 20사이클을 보냄 → 입력 검증(위반 시 422) → 스케일 변환 → Production 모델 추론 → 점수·등급·권고·버전 응답 → 기록	<code>app.predict</code> · <code>ModelManager.get</code>
③ 드리프트 감지	확정 점수 배치 → 같은 모델로 예측해 (예측, 실제) 쌍 생성 → 12시간 윈도우 RMSE → 3회 연속 5.0 초과면 DRIFT → [WARN] 알림 + 원인 센서	<code>app.batch_test</code> · <code>DriftMonitor.observe</code> · <code>diagnose</code>
④ 재학습·교체	백그라운드 스레드 → Production 가중치에서 최근 7일로 fine-tuning → 게이트 재검증 → Registry alias v2로 이동 → 서버 모델 무중단 교체 → 윈도우 초기화	<code>check_and_trigger</code> · <code>fine_tune</code> · <code>ModelManager.reload</code>

호름	지나가는 단계	담당 코드
⑤ 실패·롤백	후보가 게이트를 넘지 못하면 기록만 남기고 기존 Production 유지 → 윈도우 초기화 → 필요하면 수동 롤백	[FAIL] 로그 · <code>registry rollback</code>

3 데이터와 시간 분할

장비 5대의 3년 치 기록을 시간 순서대로 자릅니다. 무작위로 섞지 않는 이유는 미래 데이터로 과거를 맞히는 누설*5을 막기 위해서입니다. 스케일러*6와 월별 기준 분포*7는 학습 구간(2023-2024)에서 한 번만 만들고 이후 계속 재사용합니다.



* 개념 짚기

*5 시간 기반 분할과 데이터 누설

시계열은 섞지 않고 시간 순서대로 자릅니다. 미래 정보가 학습이나 입력에 섞이면 성능이 실제보다 좋아 보입니다.
 EtchGuard → 2023-24 학습, 2025 검증·테스트, 07-20 이후 드리프트. 입력에 정답 필드가 들어오면 422로 거부합니다.

*6 스케일러와 학습·서빙 일치

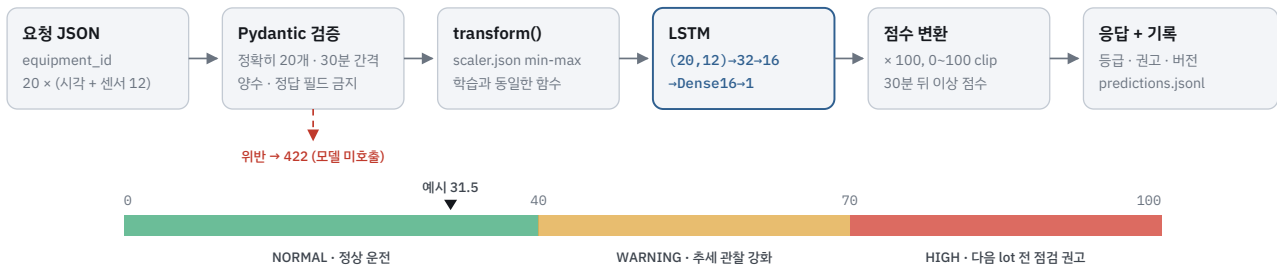
센서마다 단위가 달라 값을 0~1로 맞추는 min-max 정규화입니다. 학습과 서빙의 기준이 다르면 같은 값이 다른 숫자로 들어갑니다.
 EtchGuard → `scaler.json` 은 학습 구간에서 한 번만 만들고, 모델과 한 번들로 묶어 계속 씁니다.

*7 계절성과 같은 달 기준

매년 반복되는 정상적인 변화입니다. 전월과 비교하면 여름 같은 계절 변화를 드리프트로 착각합니다.
 EtchGuard → 외기·습도·냉각수 PSI는 같은 달 기준이면 0.28~0.46, 1년 전체를 하나로 묶은 기준이면 4.7~5.6입니다.

4 요청 하나의 여정

식각 장비*8의 최근 20사이클*9로 30분 뒤 이상 점수*10를 묻는 `/predict` 호출 하나를 따라갑니다. 모델 앞에는 입력 검증*11과 전처리가 있고, 전처리는 학습 때와 같은 `transform()` 과 같은 `scaler.json` 을 씁니다.



왜 20개를 받나. 이상 점수는 최근 20사이클 동안 쌓인 스트레스와 그 추세로 정해집니다. 지금 값이 정상 범위여도 10시간 내내 한쪽으로 움직였다면 위험합니다. 정답 필드(`anomaly_score` 등)가 입력에 섞이면 거부해 정답 누설을 막습니다.

엔드포인트 한눈에^{*12}

Method · URL	역할	주요 응답
POST <code>/predict</code>	최근 20사이클 → 30분 뒤 점수·등급·권고·모델 버전	200 · 422 · 503
POST <code>/predict/batch-test</code>	확정 점수 배치(21건 이상) → 예측 기록, 드리프트 판정, 원인 센서, 재학습 트리거	200 · 422
GET <code>/health</code> · <code>/ready</code>	프로세스 상태·모델 버전·로딩 방식 / 요청 수신 준비 여부	200 · 503
GET <code>/monitoring/status</code>	장비별 윈도우 RMSE와 판정, 재학습 이력, 엔드포인트별 지연·에러	200
POST <code>/data/upload</code> · GET <code>/data/status</code>	CSV 검증·저장(학습은 별도 명령) / 현재 데이터 현황	200 · 400 · 413
GET <code>/data/example</code> · <code>/data/batch</code>	예측 예시 입력 / 시뮬레이션용 확정 점수 배치	200 · 404
GET <code>/logs</code> · <code>/logs/{name}</code>	aiops.log, predictions.jsonl 읽기 전용 조회	200 · 400 · 404

* 개념 짚기

*8 식각(Etch)과 FDC

식각은 플라즈마로 웨이퍼 위의 막을 패턴대로 깎는 공정입니다. FDC는 센서마다 고정된 상·하한을 두고 경보를 내는 기존 감시 방식입니다. EtchGuard → 센서 하나씩이 아니라 12종을 20사이클 동안 함께 보고 점수 하나로 요약합니다.

*10 이상 점수와 위험 등급

장비 상태를 0~100으로 나타낸 값입니다. 70 이상은 HIGH, 40~70은 WARNING, 40 미만은 NORMAL입니다. EtchGuard → 현재가 아니라 30분 뒤 공정의 점수를 예측합니다.

*9 사이클과 SEQ_LEN

사이클은 장비의 공정 한 번(30분), SEQ_LEN은 모델이 한 번에 보는 과거 사이클 수입니다. EtchGuard → SEQ_LEN은 20(10시간)이고 입력 모양은 (20, 12)입니다.

*11 입력 스키마와 422

요청 형식을 Pydantic으로 정의해 두고, 맞지 않으면 모델을 부르기 전에 422로 돌려보냅니다. EtchGuard → 정확히 20개, 30분 간격의 시간순, 압력·RF 양수, 정의되지 않은 필드 금지.

※12 엔드포인트와 상태 코드

엔드포인트는 기능별 주소이고, 상태 코드는 처리 결과입니다.

200 성공, 4xx 요청 문제, 5xx 서버 문제입니다.

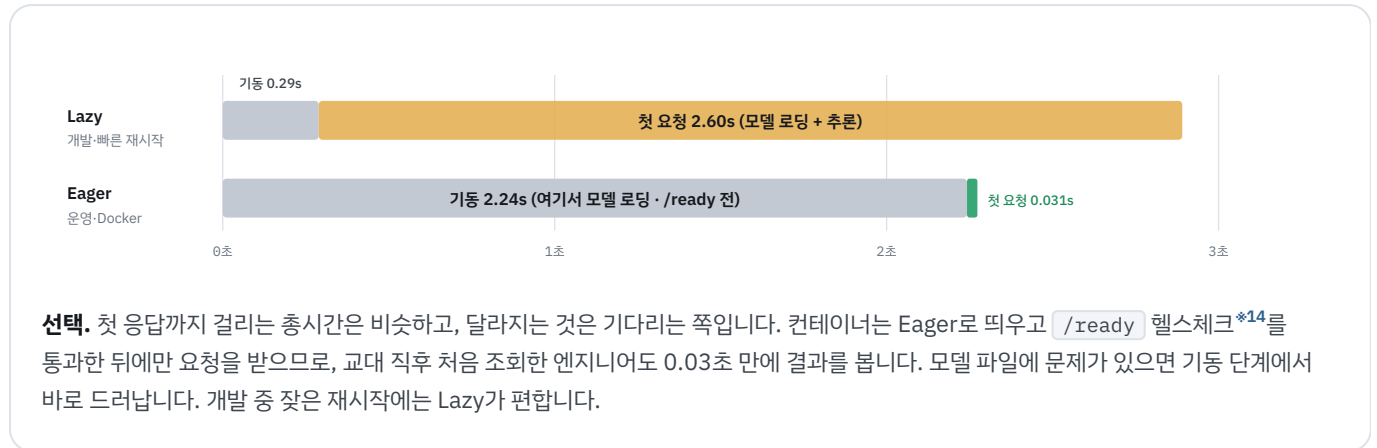
EtchGuard → 400 형식 오류, 404 없는 장비·파일, 413 64MiB

초과, 422 입력 검증 실패, 503 모델 미준비.

5 Lazy와 Eager: 로딩 시간을 누가 기다리나

모델을 메모리에 올리는 데 약 2.5초가 걸립니다. Lazy에서는 첫 사용자가, Eager에서는 서버 기동이 그 시간을 떠안습니다

※13. 아래 막대는 실제 측정값을 같은 시간 축에 그린 것입니다.



※ 개념 짚기

※13 Lazy / Eager 로딩

모델을 첫 요청 때 메모리에 올리면 Lazy, 서버가 뜰 때 올리면 Eager입니다. 로딩 시간을 누가 기다리느냐가 달라집니다.

EtchGuard → Lazy의 첫 요청은 2.60초, Eager의 첫 요청은 0.031초입니다. Docker는 Eager로 띄웁니다.

※14 /health와 /ready

`/health` 는 프로세스가 살아 있는지, `/ready` 는 모델을 들고 요청을 받을 준비가 됐는지를 확인합니다.

EtchGuard → Docker 헬스체크는 `/ready` 를 봅니다. 모델이 없으면 503입니다.

6 버전과 배포 게이트

학습할 때마다 MLflow에 새 버전이 기록되지만*¹⁵, 서버가 읽는 것은 @Production 별칭*¹⁶이 가리키는 한 버전뿐입니다. 게이트*¹⁷를 통과한 버전만 별칭을 가져갈 수 있고, 게이트는 RMSE*¹⁸와 HIGH 등급의 Precision·Recall*¹⁹로 판단합니다.



base 모델 성능

평가 구간	표본	RMSE	Precision	Recall	직전값 규칙 RMSE
검증(2025-01-01~04-11) · 게이트 판정	24,070	3.164	0.876	0.933	3.757
정상 테스트(04-11~07-20)	24,070	3.083	0.932	0.942	3.736
드리프트(07-20 14:00~12-31)	39,460	8.072	0.788	0.996	3.565

정상 구간에서는 LSTM이 직전 점수를 그대로 쓰는 규칙보다 RMSE가 17.5% 낮습니다. 드리프트 구간에서는 오히려 그 규칙보다 나빠지는데, 이것이 드리프트 감지와 재학습이 필요한 직접적인 근거입니다.



※ 개념 짚기

※15 실험 추적(MLflow Tracking)

학습할 때마다 설정(params), 성능(metrics), 결과물(artifact)을 남겨서 어떤 모델이 어떻게 만들어졌는지 추적하는 일입니다.

EtchGuard → 데이터 해시, epoch, 검증·테스트·드리프트 지표를 실험 `EtchGuard-AI0ps`에 남깁니다.

※17 배포 게이트

정해 둔 품질 기준을 넘은 모델만 운영에 올리는 관문입니다. 통과하지 못한 모델도 기록은 남습니다.

EtchGuard → 검증 RMSE 4.0 이하, HIGH Recall 0.80 이상, Precision 0.80 이상.

※19 Precision과 Recall

Precision은 HIGH 경보 가운데 실제로 HIGH였던 비율이고, Recall은 실제 HIGH 가운데 경보로 잡아낸 비율입니다.

EtchGuard → Recall이 낮으면 불량 웨이퍼가 진행되고, Precision이 낮으면 멀쩡한 장비를 세웁니다.

※16 레지스트리·버전·alias

레지스트리는 등록된 모델의 버전 목록이고, alias는 특정 버전을 가리키는 이름표입니다.

EtchGuard → 서빙은

`models:/Semiconductor_Etch_LSTM@Production` 만 읽으므로 버전이 늘어도 서빙 코드는 바뀌지 않습니다.

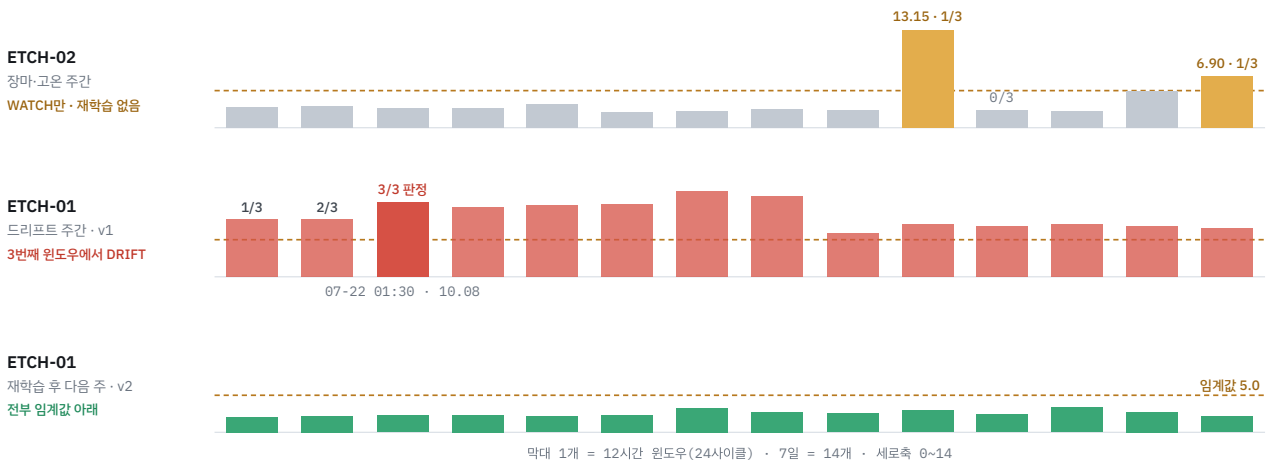
※18 RMSE

예측과 실제의 차이를 제곱해 평균을 낸 뒤 제곱근을 씌운 값입니다. 큰 오차에 더 민감하고 단위가 점수와 같습니다.

EtchGuard → $\sqrt{\text{mean}((\text{실제}-\text{예측})^2)}$. 정상 테스트 3.08점, 드리프트 구간 8.07점.

7 드리프트 판정: 한 번 튕 것과 계속 나쁜 것

확정 점수가 들어오면 장비별로 (예측, 실제) 쌍을 24개(12시간)씩 묶어 RMSE를 계산합니다. 5.0을 넘는 윈도우*²⁰가 3번 연속 나오면 드리프트*²¹로 판정합니다. 1회가 아니라 3회 연속으로 정한 이유는 오탐과 미탐*²²의 균형 때문입니다. 판정은 예측 오차로 하고*²³, 입력 분포(PSI*²⁴)는 원인 센서를 짚는 데만 씁니다. 아래는 최종 실행의 실제 윈도우 값입니다.



왜 3회 연속인가. 장마 주간의 13.15는 돌발 이상 한 번이었고 바로 다음 윈도우에서 2.36으로 돌아왔습니다. 1회만 보고 판정하면 정상 운전 중에도 장비당 월 2.06회 재학습이 일어납니다. 3회 연속(36시간 지속)으로 판정하면 2025년 정상 구간의 오경보는 0회이고, 진짜 드리프트는 장비 5대 모두 36시간 안에 잡힙니다.

원인 센서는 어떻게 고르나. 배치 입력을 2023-24년 같은 달의 분포와 비교해 센서별 PSI를 구하고, 학습 기간의 평소 PSI로 나눈 배율 상위 3개를 경보에 붙입니다. 드리프트 주간 ETCH-01은 `rf_power_w × 10.2`, `vibration_mm_s × 2.6`, `chamber_pressure_mtorr × 2.4` 로, 데이터에 실제로 주입된 변화(RF↓, 진동↑, 압력↑)와 일치했습니다. 이 순위는 경보를 올리는 데 쓰지 않고 점검 순서를 안내하는 데만 씁니다.

* 개념 짚기

*20 윈도우와 연속 판정

오차를 일정 개수씩 묶어 판단하는 단위가 윈도우입니다. 짧으면 잡음에 민감하고, 길면 반응이 느립니다.

EtchGuard → 24쌍(12시간) 윈도우가 5.0을 3번 연속 넘으면 DRIFT, 1~2번이면 WATCH입니다.

*22 오탐·미탐과 알람 피로

기준을 낮추면 미탐이 줄고 오탐이 늘며, 높이면 그 반대입니다. 오탐이 잦으면 사람들이 경보를 무시하게 됩니다.

EtchGuard → 1회 판정은 정상 운전 중에도 장비당 월 2.06회 오탐, 3회 연속은 0회입니다(대신 탐지에 36시간).

*24 PSI

기준 분포와 현재 분포를 구간별 비율로 비교한 값입니다. $\sum(\text{현재} - \text{기준}) \times \ln(\text{현재} / \text{기준})$ 이며, 보통 0.25를 넘으면 크게 변했다고 봅니다.

EtchGuard → 장비 열화 때문에 정상일 때도 1을 넘기 쉬워서, 평소 PSI 대비 배율(예: ×10.2)로 원인 센서 순위만 매깁니다.

*21 데이터 드리프트와 컨셉 드리프트

입력 분포가 바뀌면 데이터 드리프트, 같은 입력에 대한 정답 관계가 바뀌면 컨셉 드리프트입니다. 둘 다 애려 없이 예측을 틀리게 만듭니다.

EtchGuard → 07-20 이후 압력↑ RF↓ 진동↑와 열화 가속이 겹쳐 RMSE가 3.08에서 8.07로 올랐습니다.

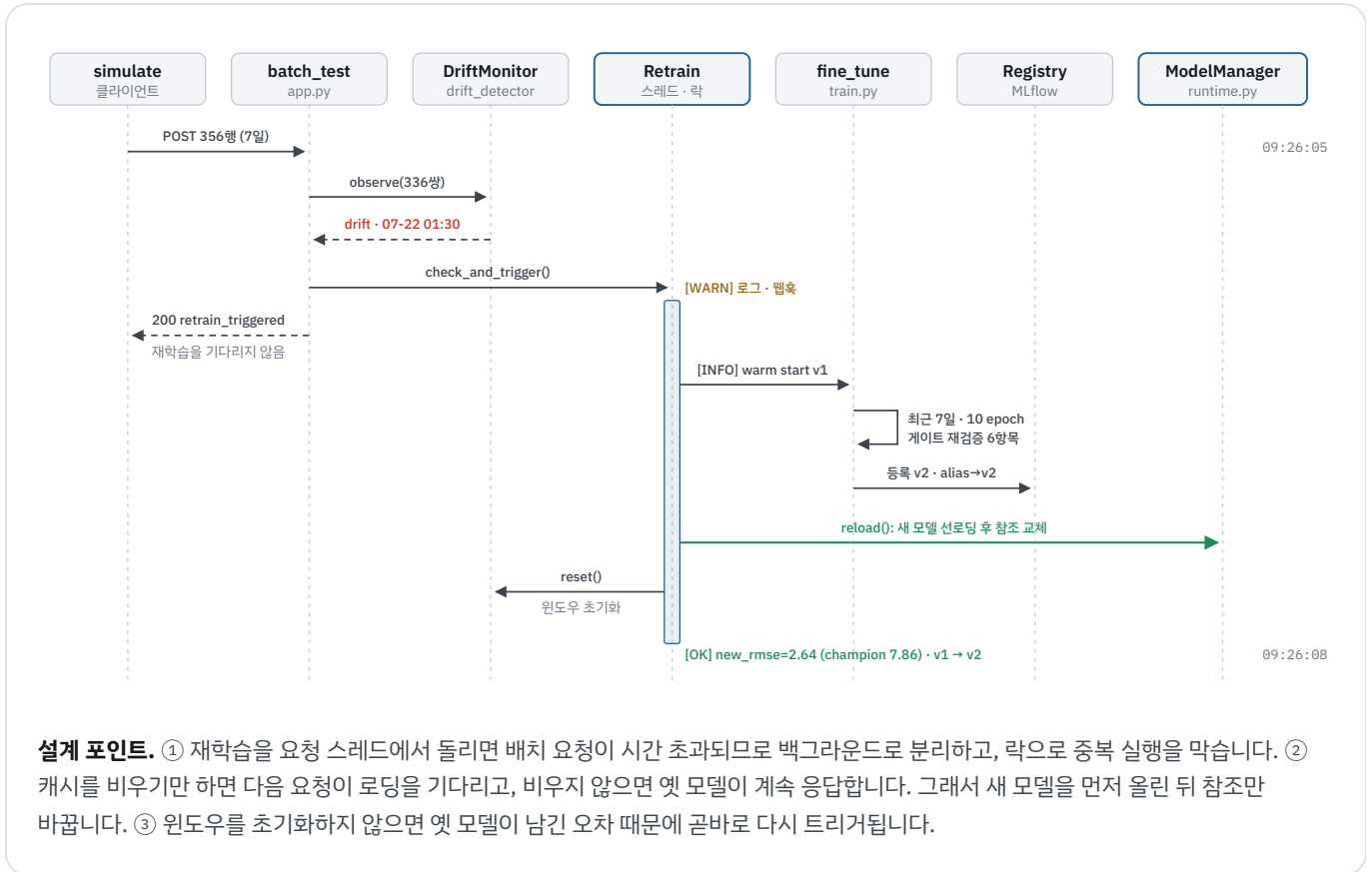
*23 성능 감시와 입력 감시

정답이 있으면 예측 오차로 직접 판정하고, 정답이 늦게 오면 입력 분포 변화로 먼저 짐작합니다.

EtchGuard → 판정은 예측 오차(RMSE)로 하고, 입력 분포(PSI)는 원인 센서를 짚는 데만 씁니다.

8 재학습 시퀀스: 3초 동안 일어나는 일

드리프트 배치를 보낸 요청은 재학습을 기다리지 않고 곧바로 200을 받습니다. 재학습은 백그라운드 스레드 *²⁵에서 돌고, 통과하면 서빙 모델을 멈추지 않고 교체합니다*²⁶. 시각은 최종 실행의 `aiops.log` 기준입니다.



재학습 데이터는 어디서 오나. 드리프트를 감지한 배치의 마지막 시각(2025-07-27 13:30)을 기준으로, 데이터 저장소(최신 업로드 CSV)에서 직전 7일 치를 장비 5대 모두 가져옵니다. 장비별로 시간 순서를 지켜 앞 80%(1,340건)로 학습하고 뒤 20%(340건)로 검증합니다. 스케일러는 base 번들의 것을 그대로 복사합니다.

* 개념 짚기

*²⁵ 백그라운드 작업과 락

오래 걸리는 작업을 요청 처리와 분리해서 돌리고, 같은 작업이 동시에 두 번 실행되지 않도록 막는 장치입니다.

EtchGuard → batch-test는 곧바로 200을 받고, 재학습 중에 들어온 다른 장비 요청은 `retrain_in_progress` 를 받습니다.

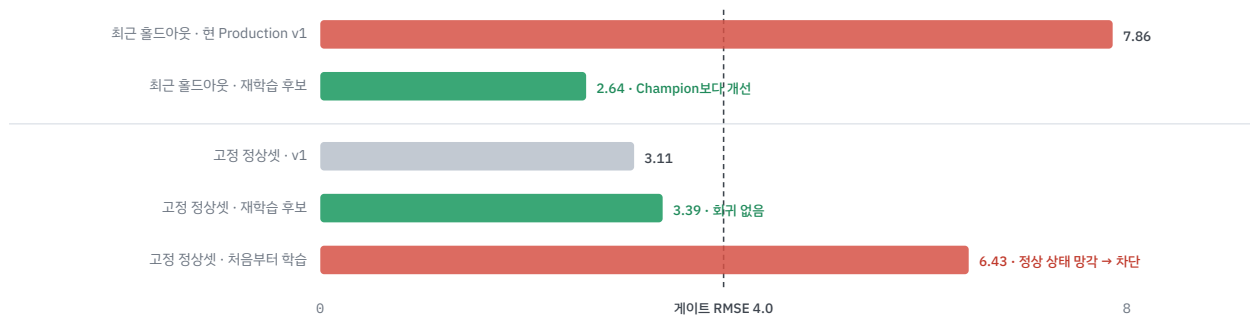
*²⁶ 무중단 교체

서비스를 멈추지 않고 모델을 바꾸는 일입니다. 캐시를 비우기만 하면 다음 요청이 로딩을 기다리고, 비우지 않으면 옛 모델이 계속 응답합니다.

EtchGuard → `reload()` 가 새 모델을 먼저 올려 둔 뒤 참조만 바꿉니다.

9 게이트 재검증: 무엇을 넘어야 교체되나

재학습 후보는 세 조건을 모두 만족해야 합니다. 최근 홀드아웃에서 기본 게이트를 넘어야 하고, 현 Production보다 나아야 하며*²⁷, 드리프트에 맞추다가 정상 상태를 잊지 않았는지*²⁸ 고정 정상셋으로 확인합니다. 하나라도 못 넘으면 기존 버전이 그대로 서비스됩니다*²⁹.



왜 warm start인가. 같은 7일 데이터로 처음부터 학습하면 시드에 따라 결과가 ± 0.36 씩 흔들리고, 정상 구간을 잃어 정상셋 RMSE가 6.43까지 나빠집니다. Production 가중치에서 이어서 10 epoch만 학습하면 편차가 ± 0.009 로 작고 정상셋도 3.39로 유지됩니다. 스케일러를 다시 맞추지 않는 이유는 같은 센서 값이 다른 숫자로 들어가면 이어서 학습하는 의미가 사라지기 때문입니다.

* 개념 짚기

*27 Champion / Challenger · 회귀 테스트

서비스 중인 모델과 후보 모델을 같은 데이터로 비교하고, 예전에 잘하던 것을 여전히 잘하는지 고정 데이터로 다시 확인하는 절차입니다.

EtchGuard → 홀드아웃 RMSE 7.86 → 2.64, 고정 정상셋 2,410건 RMSE 3.39(기준 4.0 이하).

*28 Fine-tuning · warm start · 망각

이미 학습된 가중치에서 이어서 조금 더 학습하는 방식입니다. 적은 데이터로 처음부터 학습하면 결과가 불안정하고 예전 패턴을 잊어버립니다(망각).

EtchGuard → 최근 7일 데이터로 학습률 $1e-4$, 10 epoch. 처음부터 학습하면 정상셋 RMSE가 6.43까지 나빠집니다.

*29 롤백

문제가 생겼을 때 이전 버전으로 되돌리는 일입니다. 게이트에서 떨어지면 기존 버전을 그대로 두는 것이 자동 롤백 역할을 합니다.

EtchGuard → 수동 롤백은 `registry rollback --version N`이며, 게이트 통과 이력이 있는 버전만 허용합니다.

10 운영 지표와 로그

모델 품질과 별개로 서비스 자체도 지켜봅니다. 모든 요청은 미들웨어를 거치며 엔드포인트별로 응답 시간과 결과 코드가 집계되고, `/monitoring/status`에서 확인할 수 있습니다^{*30*31}.

엔드포인트	요청	p50	p95	4xx	5xx
<code>/predict/batch-test</code> (7일 배치)	20	35 ms	105 ms	0	0
<code>/data/batch</code>	20	6 ms	7 ms	0	0
<code>/predict</code>	1	54 ms	54 ms	0	0
<code>/monitoring/status</code>	22	1 ms	27 ms	0	0

운영 목표는 `/predict` p95 1초 미만, 5xx 0%입니다. 최종 실행(대시보드 시연 포함)에서 5xx는 한 건도 없었습니다.

logs/aiops.log · 사람이 읽는 운영 로그

```
[WARN] drift detected equipment=ETCH-01
      at=2025-07-22T01:30:00 window_rmse=10.08
      > 5.0 x3 | suspect: rf_power_w(x10.2) ...
[INFO] retrain triggered (warm start from
      production-v1, data=last 7d ...)
[OK]   new_rmse=2.64 (champion 7.86) ...
      production-v1 → production-v2
```

logs/predictions.jsonl · 모든 예측의 기록

```
{"logged_at": "2026-10-01T09:26:05",
 "source": "batch-test",
 "equipment_id": "ETCH-01",
 "target_timestamp": "2025-07-21T01:30:00",
 "predicted": 86.786, "actual": 96.168,
 "model_version": "production-v1"}
```

한 줄에 예측 하나씩 기록하므로, 어느 버전이 언제 어떤 값을 냈는지 나중에 추적하고 품질 문제를 역으로 따라갈 수 있습니다.

`ALERT_WEBHOOK_URL` 을 설정하면 [WARN]·[ERROR]가 사내 메신저 웹훅으로도 전송됩니다.

* 개념 짚기

*30 p50 · p95 지연시간

요청 응답 시간을 줄 세웠을 때 50%, 95% 지점의 값입니다.
평균은 몇 건의 느린 요청을 가려 버리므로 p95로 체감 품질을 봅니다.
[EtchGuard](#) → 최근 1,000건 기준. 7일 배치 판정 p50 35ms, p95 105ms.

*31 처리량과 에러율

처리량은 일정 시간 동안 처리한 요청 수, 에러율은 그중 실패한 비율입니다. 4xx는 요청 쪽, 5xx는 서버 쪽 문제라 따로 셉니다.
[EtchGuard](#) → 엔드포인트별 요청 수와 4xx·5xx 건수를 집계하고, 최종 실행의 5xx는 0건이었습니다.

11 Docker: 빌드할 때와 실행할 때

이미지^{*32}를 만들 때 학습, 등록, 승격까지 끝내 두므로 컨테이너는 뜨자마자 Production 모델을 들고 시작합니다. 호스트의 MLflow DB를 복사하지 않아 절대경로 문제가 없고, 의존성을 소스보다 먼저 설치해 레이어 캐시^{*33}를 활용합니다.



단일 컨테이너인 이유. 실습 범위에서 별도 MLflow 서버와 모니터링 스택은 제외했습니다. MLflow는 컨테이너 안의 SQLite와 로컬 artifact로 동작하고, 서빙과 재학습이 같은 파일 시스템을 씁니다. 컨테이너 안에서도 드리프트 → 재학습 → v1에서 v2 교체까지 똑같이 재현됐습니다.

* 개념 짚기

*32 이미지와 컨테이너

이미지는 코드·라이브러리·모델을 묶은 실행 꾸러미이고, 컨테이너는 그 이미지를 실제로 띄운 실행 단위입니다. 어디서 실행해도 환경이 같습니다.
[EtchGuard](#) → 빌드할 때 학습과 등록을 끝내 두고, 실행하면 Eager로 바로 서빙합니다.

*33 레이어 캐시

Dockerfile의 각 줄이 층으로 저장되고, 바뀌지 않은 층은 다시 실행하지 않는 기능입니다.
[EtchGuard](#) → 의존성을 먼저 설치하므로 코드만 고치면 TensorFlow를 다시 설치하지 않습니다.

12 코드 지도와 직접 실행하기

그림의 각 단계가 어느 파일의 어떤 함수인지, 실습 가이드의 TODO와 어떻게 이어지는지 정리했습니다.

단계	파일 · 함수	가이드 TODO	하는 일
업로드·검증	<code>data.read_data</code> · <code>app.upload</code>	—	필수 열, 30분 간격, 장비별 연속성, 값 범위 검증 후 저장
분할·스케일러	<code>data.partitions</code> · <code>fit_scaler</code>	—	시간 순 분할, 학습 구간으로만 min-max fit
학습·게이트·등록	<code>train.train_base</code> · <code>deployment_gate</code> · <code>register_bundle</code>	—	LSTM 학습, 게이트 판정, MLflow 기록, 통과 시 alias 이동
모델 로딩	<code>ModelManager._load_from_mlflow</code>	<code>_load_from_mlflow()</code>	<code>@Production</code> 버전을 번들째 로드
배치 판정	<code>app.batch_test</code>	<code>batch_test()</code>	20개 슬라이딩 윈도우 예측, 쌍 누적, 기록
RMSE	<code>drift_detector.compute_rmse</code> · <code>is_drift</code>	<code>compute_rmse()</code>	12시간 윈도우 RMSE, 3회 연속 판정
트리거	<code>retrain_trigger.check_and_trigger</code>	<code>check_and_trigger()</code>	[WARN] 알림, 백그라운드 재학습, [OK]·[FAIL], 윈도우 초기화
재학습·교체	<code>train.fine_tune</code> · <code>ModelManager.reload</code>	—	warm start, 게이트 재검증 6항목, 무중단 교체
원인 센서	<code>data_drift.diagnose</code>	—	같은 달 기준 PSI ÷ 평소 PSI 상위 3개
시뮬레이션	<code>scripts/simulate_drift.send_batch</code>	<code>send_batch()</code>	정상 → 계절 → 드리프트 → 재학습 후 배치 전송
롤백	<code>registry.py rollback</code>	—	통과 이력이 있는 버전으로만 alias 되돌리기

그림을 직접 다시 만들어 보기

확인할 것	명령 (프로젝트 루트)
학습·게이트 ·Production v1	<code>python -m semiconductor.train --register</code>
서버 기동(운영 설정)	<code>MODEL_SOURCE=mlflow LOADING_MODE=eager uvicorn semiconductor.app:app --port 8000</code>
5절 Lazy·Eager 측정	<code>python -m semiconductor.verify --spawn --mode lazy --label local_lazy</code> (eager도 같은 방식)
7·8절 드리프트 → 재학습	<code>python scripts/simulate_drift.py</code>
7절 판정 기준 표	<code>python scripts/analyze_drift_policy.py</code>
9절 warm start 비교	<code>python scripts/compare_retrain_strategy.py</code>
6절 게이트 실패·버전 이력	<code>python -m semiconductor.train --register --bundle <번들> --rmse-gate 0</code> · <code>python -m semiconductor.registry list</code>
전체 자동 테스트 (11개)	<code>python -m unittest semiconductor.test_pipeline -v</code>
11절 Docker	<code>docker compose up --build -d</code>

13 한계

한계	영향	다음 단계
합성 데이터(HIGH 비율 20%로 보정)	실제 팜에서의 성능은 주장할 수 없습니다.	실제 FDC·계측 데이터로 다시 검증
확정 점수가 즉시 도착한다고 가정	실제로는 계측 결과가 몇 시간 늦게 옵니다.	정답이 늦는 구간에는 PSI 진단을 선행 경보로 쓰는 정책 검토
감시 상태를 프로세스 메모리에 보관	워커가 여러 개이거나 재기동하면 윈도우가 초기화됩니다.	Redis 같은 공유 저장소, Prometheus·Grafana로 이전
인증 없음, 웹훅은 선택 설정	사내망 PoC 범위에서만 쓸 수 있습니다.	API 키·SSO, 사내 메신저 연동, 감사 로그
원인 센서 기준 분포가 과거 정상에 고정	새 레시피가 자리 잡아도 RF 파워가 계속 상위에 나옵니다.	승격할 때 기준 분포 갱신을 엔지니어가 승인